FLOW STATISTICS-BASED ANOMALY DETECTION IN SDN-IOT NETWORKS

Group - 9

Kiranmai - CB.SC.U4AIE24119
Sudeshna - CB.SC.U4AIE24120
P. Myagi Ranganadh - CB.SC.U4AIE24143
Devisri - CB.SC.U4AIE24163

INTRODUCTION

- The Internet of Things (IoT) connects many smart devices, but these networks often have weak security and are vulnerable to cyber-attacks. Traditional security methods cannot effectively monitor and protect large IoT networks.

- Software Defined Networking (SDN) provides centralized control and better visibility of network traffic. In this project, we use SDN to continuously monitor flow statistics in the network. The packet rate of each flow is analyzed to detect abnormal traffic behavior.

- Based on predefined threshold values, traffic is classified into different severity levels. When high abnormal traffic is detected, the system automatically blocks the malicious flow and unblocks after some time. This helps prevent network congestion and improves IoT network security and performance.

PROBLEM STATEMENT

IoT networks are highly vulnerable to cyber-attacks due to limited device resources and weak security mechanisms.

Traditional security approaches cannot provide real-time monitoring and rapid response in large-scale IoT environments.

Hence, there is a need for an SDN-based centralized solution that can continuously analyze network flow statistics to detect anomalies and automatically mitigate malicious traffic.

OBJECTIVES

- To design an SDN-based IoT network
- To monitor IoT traffic using flow statistics
- To classify anomalies based on severity
- To apply adaptive control actions using SDN

HOW TO IMPLEMENT THIS IDEA?

SIMULATORS USED :

Mininet

CONTROLLER :

(RYU)SDN controller

Architecture Diagram :

```
+------------------+
|   Ryu Controller |
|  (Anomaly Detection) |
+---------+--------+
          |
          | OpenFlow (Control Messages)
          v
    +------------+
    |  Switch 1  |
    | Flow Table |
    +---+---+---+
        |   |   |
        |   |   |
       H1  H2  H3
    (Host)(Host)(Host)
```

What is SDN?

Software-Defined Networking (SDN) is a network architecture that separates the control plane from the data plane, enabling centralized and programmable control of network behavior through a software-based controller.

In our project, SDN is implemented using the Ryu controller . The SDN controller acts as the central control unit that continuously monitors IoT traffic using flow statistics collected from the switches. Based on the observed traffic behavior, the controller detects abnormal traffic and dynamically applies control actions such as rate limiting or blocking to maintain network stability.

METHODOLOGY :

Step - 1: IoT Network Setup

- IoT devices are connected via OpenFlow switches
- An SDN controller manages the entire network

Step - 2: Traffic Monitoring

SDN controller periodically collects:

- Source IP
- Destination IP
- Port number
- Packet count
- Flow rate
- Flow duration

Structure of a Flow Table Rule

Every flow rule consists of:

1. Match → Condition to identify packets
2. Priority → Determines rule precedence
3. Action → Operation applied to matched packets

What is Match?

Match defines the packet characteristics required for a rule to apply.
It checks:

- Source IP
- Destination IP
- MAC Address
- Port Number

If packet satisfies match condition → rule is applied.

What is Priority?

Priority determines which rule is applied first when multiple rules match.
Higher number → Higher priority

What is Action?

Action defines what the switch should do with matched packets.
Common actions:

- Send to controller
- Forward to port
- Drop packet

How Flow Table Works ?

- Packet enters switch
- Switch checks Flow Table
- Compares packet header with Match fields
- If match found → Apply action
- If no match → Table-Miss Rule is triggered

Table-Miss Rule

- Match = Empty (matches everything not matched before)
- Priority = 0 (lowest)
- Action = Send packet to controller
- Used for:
  - MAC learning
  - Installing new flow rules

What is ARP?

- ARP is a protocol used to:
  - Find the MAC address corresponding to a given IP address.

- For example,

Device A wants to send data to Device B having IP address 10.0.0.5.
But Device A does not know the MAC address of that IP.

Device A sends a broadcast message:
“Who has IP 10.0.0.5? Tell me your MAC address.”

Device B has IP 10.0.0.5.
 It replies directly to Device A:
“IP 10.0.0.5 belongs to MAC BB:BB.”

Now Device A stores:
10.0.0.5 → BB:BB

What is MAC Learning?

- MAC Learning is the process where:
  - The switch (or SDN controller) learns which MAC address is connected to which port.

- For example,

Device A is connected to Port 2 of Switch S1.
Device A wants to send a packet to Device B.

- At this point, the switch does not know which port MAC addresses are connected to.

Device A sends a data packet:

- Source MAC = AA:AA
- Destination MAC = BB:BB

The packet enters the switch through Port 2 and This storing process is called MAC Learning.

If port of BB:BB is unknown → switch floods the packet to all ports except Port 2
Device B has MAC BB:BB
It replies directly to Device A:
“ MAC BB:BB. belongs to port 4”

Step - 3 : Suspicious Flow Detection

- Normal traffic thresholds are predefined
- If a flow shows an abnormal increase in packet rate compared to normal IoT traffic's flow rate , it is marked as blocked and it is blocked temporarily.

Step - 4 : Severity Classification

Normal Severity :

- flow rate <20

Low Severity : flow rate>20 and <50

- forwards packets.

Medium Severity : flow rate >50 and <100

- forward packets.

High Severity : flow rate>100

- Blocks the devices temporarily.
- Indicates a possible security attack

Results

```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.227 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.119 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.121 ms
^C
--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2029ms
rtt min/avg/max/mdev = 0.119/0.155/0.227/0.050 ms
```

```
🔌 Switch Connected Successfully!

------------------------------------
🕒 Time: 18:47:21
📤 Source: 10.0.0.1
📥 Destination: 10.0.0.2
⚡ Packet Rate: 0.00 packets/sec
🚦 Severity Level: NORMAL
------------------------------------

------------------------------------
🕒 Time: 18:47:21
📤 Source: 10.0.0.2
📥 Destination: 10.0.0.1
⚡ Packet Rate: 0.00 packets/sec
🚦 Severity Level: NORMAL
------------------------------------
```

```
64 bytes from 10.0.0.2: icmp_seq=40266 ttl=64 time=0.004 ms
64 bytes from 10.0.0.2: icmp_seq=40267 ttl=64 time=0.004 ms
64 bytes from 10.0.0.2: icmp_seq=40268 ttl=64 time=0.004 ms
64 bytes from 10.0.0.2: icmp_seq=40269 ttl=64 time=0.005 ms

--- 10.0.0.2 ping statistics ---
105806 packets transmitted, 105806 received, 0% packet loss, time 1287ms
rtt min/avg/max/mdev = 0.002/0.004/3.847/0.013 ms, ipg/ewma 0.012/0.006 ms
```

```
--------------------------------------------------
🕒 Time: 18:48:51
📤 Source: 10.0.0.1
📥 Destination: 10.0.0.2
⚡ Packet Rate: 21161.20 packets/sec
🚫 High Severity Device Temporarily Blocked!

🚦 Severity Level: HIGH
🔔 Total Alerts: 1
--------------------------------------------------


--------------------------------------------------
🕒 Time: 18:48:51
📤 Source: 10.0.0.2
📥 Destination: 10.0.0.1
⚡ Packet Rate: 21161.20 packets/sec
🚫 High Severity Device Temporarily Blocked!

🚦 Severity Level: HIGH
🔔 Total Alerts: 2
--------------------------------------------------
```

CONCLUSION

This project demonstrates how Software-Defined Networking (SDN) can be used to centrally monitor and control IoT network traffic. By observing simple flow statistics, the system identifies abnormal behavior and classifies it into Normal, low, medium, or high severity levels. Based on the severity, appropriate actions such as forwarding and blocking temporarily.

THANK YOU